

1.

A search scenario in an IT application can be seen in NPC pathfinding in a video game. When a player moves through a game world, an NPC may need to find a path from its current position to a specific destination. The game represents the environment as a collection of locations or nodes connected by possible paths. The search algorithm examines these possible paths and determines which one can lead the NPC to its destination. For example, an NPC may need to travel from a village to a castle while avoiding obstacles such as walls, buildings, or dangerous areas. The search process starts from the NPC's current location and explores possible routes until it finds the destination. Different search algorithms can produce different results depending on how they explore the available paths. Efficient searching is important because games may contain large environments and many possible routes. Using an appropriate search strategy allows NPCs to move realistically while reducing the processing required by the computer.

2.

A* is a search algorithm commonly used in AI for finding efficient paths between a starting point and a destination. It combines the advantages of Uniform Cost Search and Greedy Best-First Search. A* uses two main values: the actual cost of reaching a node and an estimated cost from that node to the goal. These values are combined to decide which node should be explored next. This allows A* to search intelligently instead of exploring every possible path equally. A common example is NPC pathfinding in video games, where A* can help characters find a short route around obstacles. It is also used in robotics, maps, and navigation systems. When an appropriate heuristic is used, A* can efficiently find an optimal path.

3.

I implemented Breadth-First Search (BFS) in Python using a small graph. BFS starts from a selected node and explores all of its immediate neighbours before moving to nodes at the next level. I observed that the algorithm uses a queue to keep track of which nodes should be visited next. For example, if the starting node is A, BFS first visits A's direct neighbours before continuing deeper into the graph. This makes BFS useful for finding the shortest path in an unweighted graph because it explores nodes according to their distance from the starting point. The activity helped me understand how search algorithms systematically explore information rather than randomly selecting a path. I also observed that BFS can use

more memory than some other methods because it must keep multiple nodes in the queue while searching.

Python Code — Breadth-First Search (BFS)

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

def bfs(start):
    visited = []
    queue = deque([start])

    while queue:
        node = queue.popleft()

        if node not in visited:
            visited.append(node)
            queue.extend(graph[node])

    return visited

print(bfs('A'))
```

Output:

```
['A', 'B', 'C', 'D', 'E', 'F']
```

4.

The search-tree diagram shows how Breadth-First Search (BFS) explores a graph level by level. The starting node is placed at the top of the tree, with its neighbouring nodes placed on the next level. BFS visits the starting node first and then visits each of its immediate children before moving to the next level. For example, starting from node A, the algorithm first visits A, followed by B and C. It then moves deeper and visits D, E, and F. Arrows show the connections between the nodes and demonstrate the direction in which the search progresses. The visit order can be labelled on each node to make the process clear. This diagram helps explain why BFS is useful for finding the shortest path in an unweighted graph, since it explores nodes based on their distance from the starting point.

BFS Search Tree (Visit Order)

